2026

User Guide

SAE 203

Written by Anthony MERLIN, Alexandre SELLA & Yanïs CHERIF

Table of Contents

Table of Contents....1
Introduction....2
Choice of Distribution....2
Choice of Software Stack....3
Installation, Configuration, and Deployment Steps....4
1. Installation Phase....4
2. Configuration Phase....5
3. Deployment Phase....7
4. Security and Robustness Settings....9
5. Validation Tests....10
Conclusion....11



Introduction

This report details the deployment of the YAASystem web infrastructure, carried out as part of SAÉ S2.03. The objective is to set up a WordPress CMS on a Raspberry Pi 2. This hardware has significant limitations, with only 1 GB of RAM and an 8 GB microSD card. The system must also be able to operate seamlessly and without reconfiguration, whether on a home network or on the IUT's infrastructure.

Choice of Distribution

For this project, our choice naturally fell on the Raspberry Pi OS Lite distribution. This decision is primarily driven by a strict need for resource optimization. Indeed, since the machine is limited to 1 GB of RAM and an 8 GB SD card, installing a desktop-style graphical interface was out of the question. The Lite version has the advantage of taking up very little space (less than 2 GB), which leaves us ample room to host the database and WordPress-related files. Furthermore, the absence of a graphical environment allows us to dedicate the full processing power and RAM to essential web services. Beyond hardware constraints, this choice is part of a truly professional approach. It allows us to simulate real-world production conditions in an industrial environment, where servers are typically managed via the command line using SSH.

Choice of Software Stack

Regarding the software architecture, we decided to deploy a LEMP stack, comprising Linux, Nginx, MariaDB, and PHP. The choice of web server proved particularly crucial. We preferred Nginx over the more traditional Apache solution because of its asynchronous connection handling. Whereas Apache tends to create a resource-intensive process for each new visitor, Nginx manages to maintain good performance even under load. On a machine like the Raspberry Pi, this prevents RAM from becoming saturated and stops the system from using the SD card's swap file. This is a critical factor, as using the swap file would significantly slow down the server while prematurely wearing out the storage medium. For dynamic code management, we paired Nginx with PHP-FPM. This process manager allows PHP scripts to be processed efficiently and in isolation by communicating directly with the web server via a Unix socket. Finally, we completed this infrastructure by choosing MariaDB, a database system that has become the go-to choice for its lightweight nature and perfect compatibility with WordPress on ARM architectures.

Installation, Configuration, and Deployment Steps

1. Installation Phase

This phase involves setting up the operating system and installing the software packages necessary for the web server to function.

1.1. System Preparation and Network Configuration

To ensure the server is reliably accessible and can communicate properly on the IUT network, the default dynamic addressing was not suitable. We therefore had to switch the Raspberry Pi to manual configuration (static IP) by directly modifying the network configuration files.

```
# Editing the network configuration file
sudo nano /etc/dhcpcd.conf

# Adding static addressing parameters (IUT example):
interface eth0
static ip_address=10.103.101.105/24
static routers=10.103.101.1

# Apply settings
sudo reboot
```

1.2. Installing the LEMP Stack

Once the system is up to date, we install the software foundations with a single command: the web server (Nginx), the database manager (MariaDB), and the PHP interpreter.

```
sudo apt install nginx mariadb-server php-fpm php-mysql -y
```

2. Configuration Phase

This step involves configuring the newly installed software so that it communicates with each other and is optimized for the Raspberry Pi.

2.1. Creating the Database

We create an isolated environment in MariaDB specifically for the future WordPress site, with a dedicated user.

```
sudo mariadb

CREATE DATABASE bd_yaasystem;
CREATE USER 'yaa-admin'@'localhost' IDENTIFIED BY 'our_password';
GRANT ALL PRIVILEGES ON bd_yaasystem.* TO 'yaa-admin'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

2.2. General Nginx Optimization

In the main configuration file /etc/nginx/nginx.conf, we disable the display of the Nginx version to avoid disclosing technical information (basic security).

```
http {

        ##
        # Basic Settings
        ##

        sendfile on;
        tcp_nopush on;
        types_hash_max_size 2048;
        server_tokens off; # Recommended practice is to turn this off
        client_max_body_size 64M;
```

2.3. Virtual Host Configuration (Server Block)

Next, configure how Nginx will handle the site by editing the file /etc/nginx/sites-available/default. Enable PHP processing and WordPress permalink handling using the try_files directive.

```
server {
        listen 80 default_server;

        root /var/www/html;

        index index.html index.htm index.nginx-debian.html;

        server_name _;

        location / {
                # First attempt to serve request as file, then
                # as directory, then fall back to displaying a 404.
                try_files $uri $uri/ /index.php?args;
        }

        # pass PHP scripts to FastCGI server
        #
        location ~ \.php$ {
                include snippets/fastcgi-php.conf;
        #
        #       # With php-fpm (or other unix sockets):
                fastcgi_pass unix:/run/php/php7.4-fpm.sock; # Relie Nginx à PHP
        }

        # deny access to .htaccess files, if Apache's document root
        # concurs with nginx's one
        #
        #location ~ /\.ht {
        #       deny all;
        #}
}
```

Once configured, we check the syntax and restart the service to apply the changes:

```
sudo nginx -t
sudo systemctl restart nginx
```

3. Deployment Phase

This is the final phase of deploying the WordPress application into production to make it accessible and functional.

3.1. Deploying WordPress Files

Download the latest version of WordPress and place its files in the web server's root directory, ensuring that the Nginx user (www-data) has the necessary permissions for these files.

```
cd /var/www/html/
sudo wget https://fr.wordpress.org/latest-fr_FR.tar.gz
sudo tar -xzvf latest-fr_FR.tar.gz
sudo mv wordpress/* .
sudo rm -rf wordpress latest-fr_FR.tar.gz
sudo chown -R www-data:www-data /var/www/html/
```

3.2. Configuring the wp-config.php file (Network Agnosticism)

This is the key step in the deployment: to ensure the site automatically adapts to the network the Raspberry Pi is connected to (IUT or home), we make the URLs dynamic by using the $_SERVER['HTTP_HOST'] variable.

```
// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'bd_yaasystem' );

/** Database username */
define( 'DB_USER', 'yaa-admin' );

/** Database password */
define( 'DB_PASSWORD', 'admin' );

/** Database hostname */
define( 'DB_HOST', 'localhost' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8mb4' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );


define( 'WP_HOME', 'http://' . $_SERVER['HTTP_HOST'] );

define( 'WP_SITEURL', 'http://' . $_SERVER['HTTP_HOST'] );
```

Note: If the database installation was completed via the web interface instead of the command line, the database connection form will look like this:



WordPress creation form after configuring the config file

4. Security and Robustness Settings

4.1. Firewall (UFW)

We have secured access to the Raspberry Pi:

```
sudo ufw default deny incoming
```

Block everything by default.

```
sudo ufw allow 22/tcp
```

SSH access for administration.

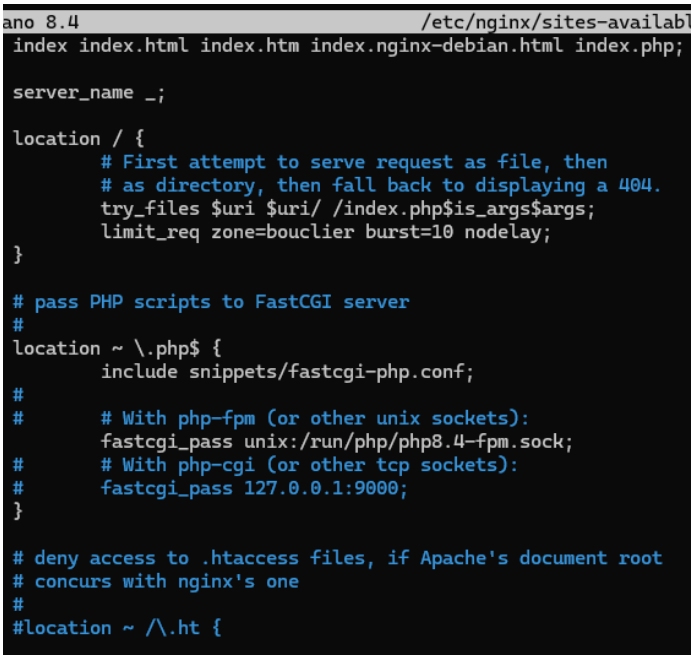
```
sudo ufw allow 80/tcp
```

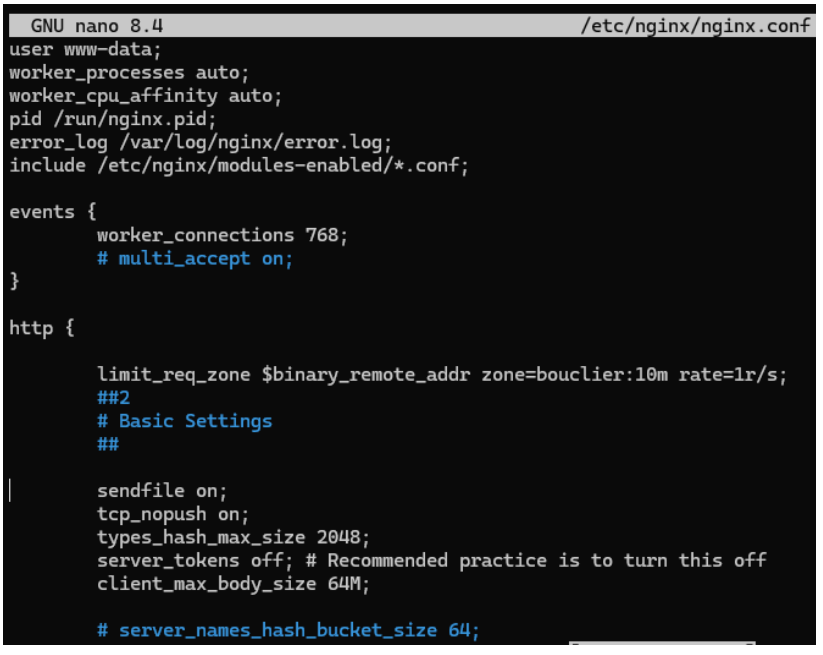
HTTP access for the website.

4.2. Anti-DoS Protection

To prevent the CPU from becoming overloaded during an attack or traffic spike, we have configured a rate limiting zone in Nginx:

- **Zone:** 10 MB of memory allocated to track incoming IP addresses.
- **Rate:** Strictly limited to **1 request per second** (`rate=1r/s`) to block attacks.
- **Tolerance:** A queue of 10 instant requests (`burst=10 nodelay`) to allow legitimate users to load the various images on a web page smoothly without being blocked by mistake.





5. Validation Tests

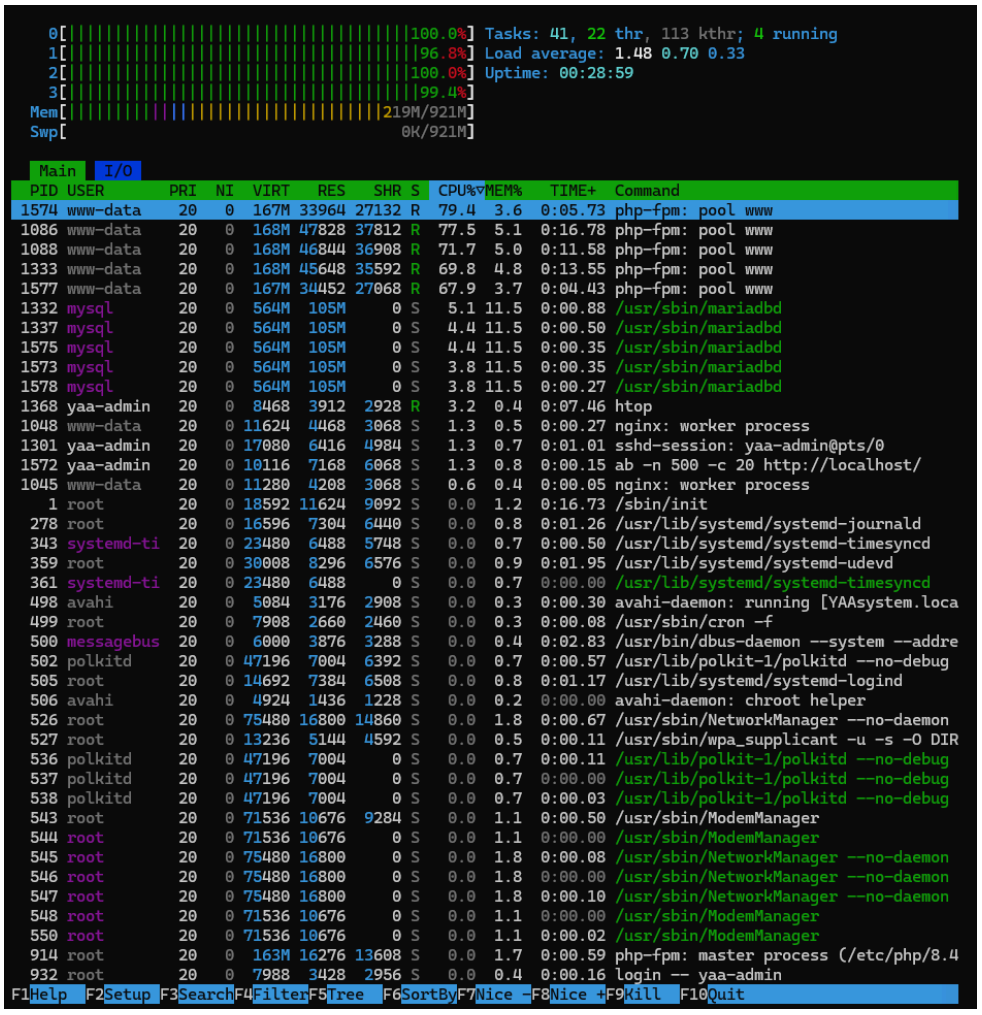
5.1. Load Test (Performance)

We used the Apache Benchmark tool to simulate 20 concurrent users making 500 requests.

Result: 2.81 requests per second processed.

htop observations: CPU usage spiked to 100% (bottleneck in PHP processing), but RAM usage remained stable at 219 MB. No requests failed.

5.2. Robustness Test



Initially, we had configured the rate limit to 5 requests per second.

By enabling the rate limit, we observed that the server rejects attempts to overload the system by exceeding 5 requests per second.

Result : the server returns a 503 error (Service Unavailable) in case of excessive requests, thus protecting the processor and ensuring the Raspberry Pi remains accessible for administration.

Our action: We reconfigured the limit to 1 request per second, as described in section 4.2, for increased security while maintaining functionality and operational readiness.

Conclusion

The deployment of the YAASystem infrastructure is a success that demonstrates it is possible to maintain a modern, dynamic web service, such as WordPress, on hardware with extremely limited resources. By adopting a minimalist approach with Raspberry Pi OS Lite and optimizing each layer of the software stack using the LEMP architecture, we were able to ensure exemplary system stability. Load testing has also proven that, despite the predictable CPU saturation, RAM management remains fully under control, thereby avoiding any risk of crashes or premature wear of the storage media.

Beyond the simple technical installation, this project allowed us to implement concrete solutions to address real-world production challenges. The implementation of a network-agnostic configuration ensures the server’s portability across different environments, while the setup of firewall filtering and bandwidth throttling provides robust protection against external threats. In conclusion, this SAÉ validates the viability of low-power, secure web hosting, perfectly suited to the constraints of the initial specifications.